GuideSync — Trekking Management System

Author

Name: AVNEESH SINGH
Roll number: 22f3002042
Student email: 22f3002042@ds.study.iitm.ac.in

I am a student of the IIT Madras BS in Data Science & Applications program. GuideSync is my MAD-2 submission — a three-role trekking-management web application built with a Flask REST API and a Vue 3 single-page frontend.

AI/LLM usage

Tool used: Claude (Anthropic) via Claude Code.

AI/LLM assistance was used for boilerplate and scaffolding, Vue component generation, styling, REST route drafting, the Redis cache wrapper, CSV/HTML report content, and for reviewing and debugging code. The core design decisions, data model, business-rule logic, and understanding of every submitted line were retained and verified by me.
The breakdown of AI/LLM usage across the MAD-2 project is declared below.

Category-wise AI/LLM usage :

Category	MAD-2 %	Notes
Backend (Flask + DB)	20%	REST API model shifts the effort; fewer server-side templates designing
Frontend (HTML/JS → Vue)	30%	Bulk of the work moved to the Vue SPA architecture modifications
Styling / UI	8%	Bootstrap 5, CSS enhancing
Auth + CRUD logic	10%	Login/RBAC token logic and CRUD handlers debugging and modifying
Extras (Charts / API)	5%	Enhancing charts design

Description

GuideSync is a three-role platform (Admin, Trek Staff, Trekker) for managing adventure treks: creating and approving treks, assigning staff, opening slots, booking with strict slot guarantees, tracking history, and generating reminders/reports. The Flask backend exposes a JSON REST API consumed by a Vue 3 SPA; Redis provides caching, and Celery + Redis run the background jobs.

AI/LLM (Claude) was used across roughly the distribution shown above — heaviest on the Vue frontend and backend scaffolding, lightest on validation, testing, and the Jinja entry shell — with all core logic understood and verified by me.

Technologies used

Backend

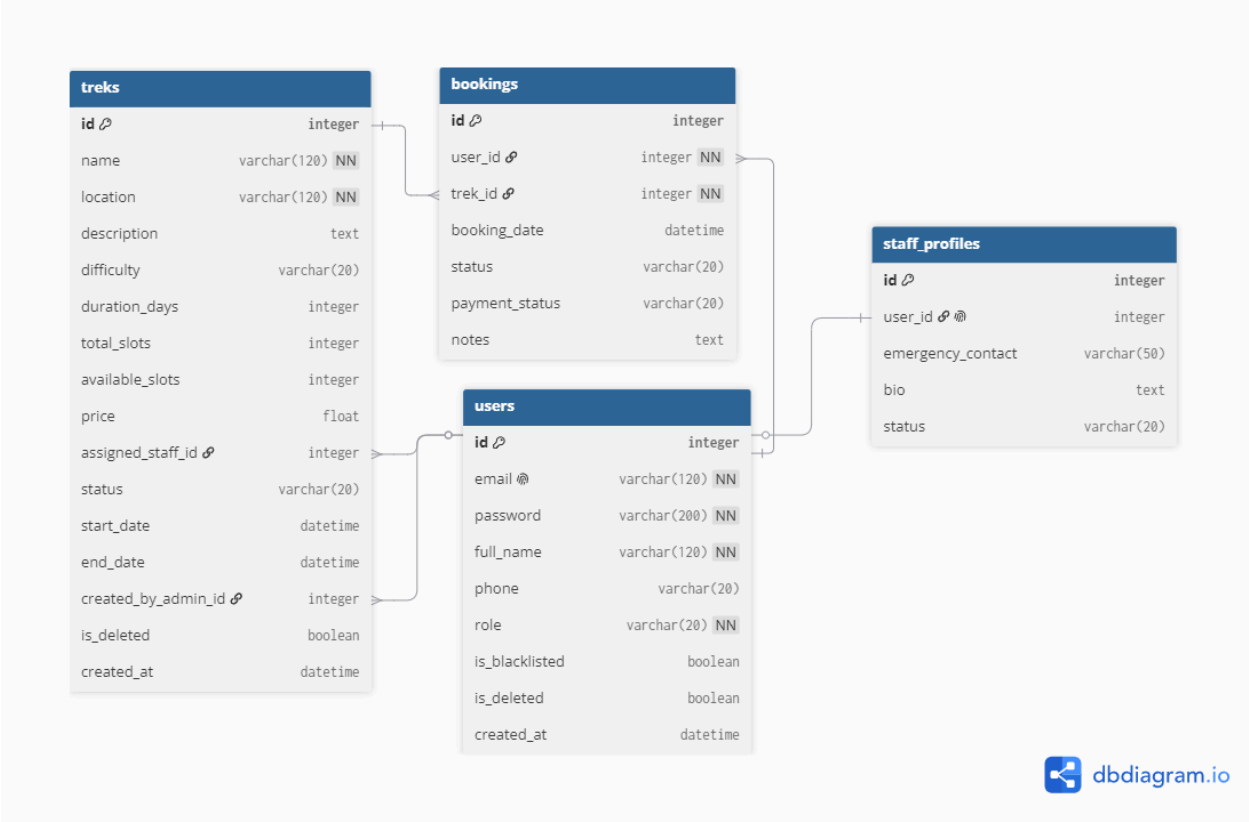
- **Flask** (app-factory pattern) — REST API layer.
- **Flask-SQLAlchemy** — ORM; database created programmatically via `db.create_all()`.
- **Flask-JWT-Extended** — token-based auth with a custom `role` claim for RBAC.
- **Flask-Caching (RedisCache)** — endpoint caching (`@cache.cached`, `@cache.memoize`).
- **Flask-CORS** — cross-origin access for the Vue dev server.
- **Werkzeug** — password hashing (`generate/check_password_hash`).
- **Celery + Redis** — background jobs (broker DB 1, result backend DB 2) and Celery Beat for cron-like scheduling.
- **Mailhog (SMTP, port 1025)** — dev mail catcher for reminders, monthly reports, and CSV export delivery.

Frontend

- **Vue 3** + **Vite** — single-page application.
- **Vue Router** — client-side routing with per-role navigation guards.
- **Pinia** — auth/session state store.
- **Axios** — API client with JWT header injection.
- **Bootstrap 5** — styling (no other CSS framework).
- **Chart.js / vue-chartjs** — admin analytics charts (bar + doughnut).

Database: SQLite (the only DB used).

DB Schema Design



API Design

The API is JSON-based, organized into four Flask blueprints; every non-auth route is protected by a `@role_required` decorator that reads the JWT `role` claim. (Full contract in the accompanying YAML.)

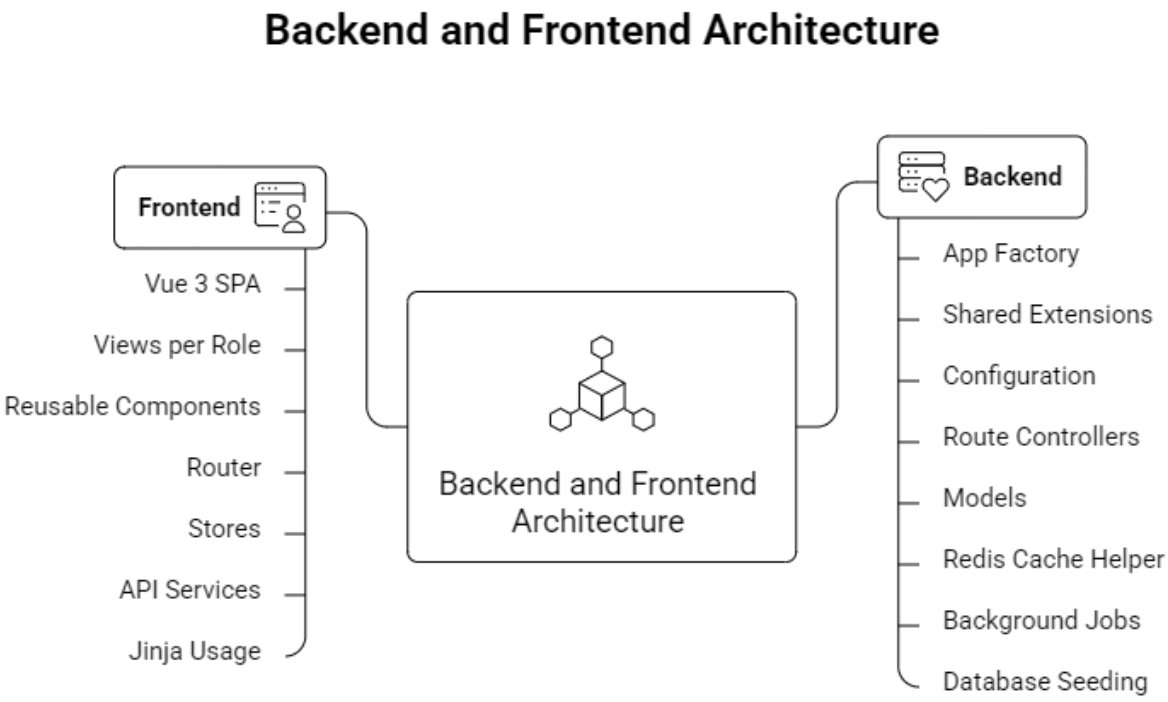
Auth (`/auth`): `POST /register` (trekker only), `POST /login`, `POST /logout`, `GET /me`.

Admin (`/admin`): `GET /dashboard`; trek CRUD `GET|POST /treks`, `PUT|DELETE /treks/<id>`, `POST /treks/<id>/assign-staff`; `GET|POST /staff`; `PATCH /users/<id>/blacklist`, `DELETE /users/<id>`; `GET /bookings`; `GET /search`; `GET /reports/analytics`, `GET /reports/summary`; background-job triggers `POST /jobs/daily-reminders`, `POST /jobs/monthly-report`, `GET /jobs/<task_id>`.

Staff (`/tstaff`): `GET /dashboard`, `GET /treks`, `PATCH /treks/<id>/slots`, `PATCH /treks/<id>/status`, `GET /treks/<id>/participants`.

Trekker (`/trekker`): `GET /dashboard`, `GET /treks` (filterable), `POST /bookings`, `PATCH /bookings/<id>/cancel`, `GET /bookings`, `PUT /profile`, `POST /export/bookings`, `GET /export/<task_id>`, `GET /export/<task_id>/download`.

Architecture and Features



Core features.
Trekker self-registration and JWT login for all three roles.
a single programmatically-seeded admin (no admin registration).
role-based access control on every protected route.
Admin: dashboard counts, trek CRUD, staff creation, staff assignment, view all bookings, global search, and moderation (blacklist / delete users and staff).
Staff: dashboard of assigned treks with participant counts, slot updates, status transitions, participant lists, mark completed.
Trekker: dashboard (available + booked treks with status), browse/filter treks, booking, cancellation with slot recovery, history, and profile edit.

Business rules are enforced server-side: no overbooking, Open-status-only booking, no duplicate active bookings, only assigned staff manage a trek, and full history preservation.
Redis caches hot endpoints with explicit invalidation on writes.
Celery runs three jobs: **daily reminders**, a **monthly HTML report** to the admin, and a **user-triggered async CSV export** — all delivered to Mailhog.

Additional features. Chart.js analytics (popular treks + booking-status charts) with a full System Report page including a live "new signups" metric and **on-demand triggers** to run the reminder/report jobs for testing.
soft-deletion of treks to protect history
a responsive Bootstrap UI.
HTML5 + backend validation and email delivery of the CSV export as an attachment.

Video

Video link - https://drive.google.com/file/d/1iJv_E4YcXVVf5SL-J3fQxV9uCWnG79AI/view